

.NET 10 与 C# 14 高级特性学习指导

以 LearnDotnetCSharp 的 53 个可运行实验为主线

LearnDotnetCSharp

2026-07-18

基于可运行实验的机制、边界与工程实践

目录

前言	5
适用读者	5
本书与仓库的关系	5
第一部分：学习环境 with 实验方法	6
1. 环境基线	6
2. CLI 使用方式	7
2.1 四类结果与进程隔离	7
2.2 正式测试、故障计划与代码 CI	7
3. 五步学习闭环	8
4. 推荐进度	8
第二部分：运行时与高级 C# 语言	8
5. 运行时基线	8
5.1 runtime.overview	8
5.2 runtime.jit-simd-pgo	9
6. C# 14 正式语言特性	10
6.1 language.csharp14	10
6.2 language.advanced	10
6.3 language.operators-conversions	10
6.4 language.generics-variance-constraints	11
6.5 language.attributes	12
6.6 language.delegates-events-iterators	12
第三部分：框架基础、集合与数据查询	13
7. 文本、时间、数值和标识	13
7.1 framework.text-globalization	13
7.2 framework.time-numerics-identity	13
8. 集合与自定义集合契约	13
8.1 collections.core-custom-buffer	13
9. LINQ: 从委托到真实 Provider	14
9.1 linq.query-semantics	14
9.2 linq.operators	14
9.3 linq.efcore-sqlite-provider	15
10. XML 与版本化序列化	15
10.1 xml.linq	15
10.2 serialization.xml-json	15

10.3 serialization.contract-versioning	15
第四部分：异步、线程与并发	16
11. 异步控制流	16
11.1 async.cancellation-stream-valuetask	16
11.2 async.task-when-each	16
11.3 async.continuations-context	17
12. 线程、执行上下文与同步	17
12.1 threading.execution-context	17
12.2 threading.synchronization-primitives	17
13. 消息传递与数据并行	18
13.1 concurrency.channel-pipeline	18
13.2 concurrency.parallel-plinq	18
13.3 concurrency.memory-model-lock-free	18
第五部分：I/O 与网络协议	19
14. 流、管道与随机访问	19
14.1 io.streams-pipelines-compression	19
14.2 io.random-access-memory-map	19
15. TCP、HTTP、UDP、WebSocket 与 TLS	19
15.1 networking.tcp-loopback	20
15.2 networking.httpclient-loopback	20
15.3 networking.http2-http3	20
15.4 networking.udp-websocket-tls	20
15.5 networking.http-streaming-resilience	21
第六部分：内存、GC 与资源生命周期	21
16. GC、Span 与所有权	21
16.1 memory.gc	21
16.2 memory.span-memory-ownership	21
16.3 memory.disposal-finalization	22
16.4 memory.pinning-native-memory	22
第七部分：诊断、反射、程序集与编译器	23
17. 可观测性与诊断	23
17.1 diagnostics.observability	23
18. 反射、插件加载与编译器	23
18.1 reflection.advanced	23
18.2 runtime.collectible-plugin	24
18.3 compiler.roslyn-il	24

18.4 compiler.incremental-generator-analyzer	25
第八部分：正则表达式与密码学	25
19. 正则表达式的复杂度边界	25
19.1 regex.advanced	25
20. 现代密码学原语	26
20.1 crypto.modern-primitives	26
第九部分：跨语言与原生互操作	26
21. 托管语言互操作	26
21.1 interop.managed-languages	26
22. 平台 P/Invoke	27
22.1 interop.native-pinvoke	27
23. Python 进程协议	27
23.1 interop.python-process-json	27
24. C ABI 与反向回调	27
24.1 interop.c-abi	27
25. C++ 不透明句柄	28
25.1 interop.cpp-opaque-handle	28
第十部分：综合训练与验收	28
26. 五个可运行综合项目	28
26.1 带 checkpoint、死信与恢复的数据管线	28
26.2 可持久化、可重启的版本化协议服务	29
26.3 多版本、可热切换的隔离插件宿主	30
26.4 可自愈的跨语言计算模块	31
26.5 可恢复的跨语言分析 workflow	32
27. 结业问答	32
28. 最终验收清单	33
附录 A：实验速查表	34
附录 B：性能实验纪律	35
附录 C：从 Markdown 生成 PDF	35
附录 D：延伸资料	36

前言

这不是一份“把 API 名字抄一遍”的手册，而是一份实验驱动的高级 C#/.NET 课程。它以 **.NET 10**、**C# 14** 和仓库中的 53 个可执行实验为基准，参考《C# 8.0 核心技术指南》的知识范围，将旧版本主题重新放到现代运行时、现代编译器和当前安全边界中理解。

完成本指导后，你应当能够回答三类问题：

1. **语言问题**：一段高级 C# 代码在类型系统、生命周期、闭包、模式、泛型、异步和表达式树层面到底表达了什么。
2. **运行时间问题**：代码编译成了什么元数据和 IL，JIT、GC、线程池、加载上下文、网络栈及密码学实现分别承担什么责任。
3. **工程问题**：怎样为取消、超时、所有权、协议、序列化版本、原生 ABI、诊断和安全建立明确边界。

学习原则：先写下预测，再运行实验；先解释语义，再谈性能；先说明所有权和失败路径，再讨论“简洁写法”。

适用读者

本指导适合已经掌握下列基础内容的学习者：

- 能编写类、接口、泛型、异常处理和基本 LINQ；
- 理解值类型与引用类型的基本差异；
- 使用过 **Task**、**async/await**，但希望建立更准确的并发模型；
- 能阅读项目文件和命令行输出；
- 愿意通过修改实验、制造失败和阅读 IL 来验证理解。

如果你刚开始学习 C#，建议先完成基础语法、面向对象和常用集合，再进入本课程。

本书与仓库的关系

本书中的每个实验都有稳定 ID，例如 **async.task-when-each**。运行器会通过反射发现所有 **IDemo** 实现，并用实验内的 **DemoAssert** 验证关键不变量。因此：

- 控制台输出用于观察机制；
- 断言用于证明实验没有悄悄偏离原目标；
- 源码才是需要逐行阅读和修改的教材；
- **self-test** 是课程整体仍然成立的回归检查。

章节与原 PDF 的对应关系见 [PDF 目录映射](#)，外部权威资料见 [官方参考资料](#)，跨语言边界的集中说明见 [互操作边界](#)。

第一部分：学习环境与方法

1. 环境基线

仓库锁定以下基线：

- SDK: **.NET SDK 10.0.301**，允许在同一策略下滚动到合适的新 feature band；
- 目标框架: **net10.0**；
- C#: **14.0**；
- Nullable: 启用；
- .NET analyzers: 启用；
- 编译器警告: 视为错误；
- Python: 工作区 **.venv**；
- C/C++: Windows x64 下使用 MSVC 构建演示库。

先运行：

```
dotnet --info
.\scripts\setup-python.cmd
dotnet restore .\LearnDotnetCSharp.slnx --ignore-failed-sources
dotnet build .\LearnDotnetCSharp.slnx
dotnet test --solution .\LearnDotnetCSharp.slnx --no-build --minimum-expected-tests 30
dotnet run --project .\src\LearnDotnetCSharp.App -- self-test
```

Windows x64 且已准备 Python、MSVC 时，预期 53 项均为 **Passed**；在 Linux 等尚未构建仓库 C/C++ 动态库的平台，受影响项目必须显示为 **Skipped**。两种环境的共同验收条件都是 **Failed = 0** 且 **Timeout = 0**，而不是把未执行的能力算作成功。

如果结果不同，不要直接跳过。按下面顺序定位：

1. 确认实际选中的 SDK 与 **global.json** 一致；
2. 确认当前目录位于仓库内；
3. 查看失败实验的异常类型和消息；
4. 单独运行失败 ID；
5. 对 Python/原生实验确认 **.venv** 与 MSVC 环境；
6. 对 HTTP/3 注意平台 QUIC 能力可能导致实验内部明确报告 **SKIPPED**，这不等于 HTTP/2 验证失败；
7. 查看 **self-test** 保存的失败子进程输出，区分断言失败、异常退出和 15 秒硬超时。

2. CLI 使用方式

```
# 全部实验
dotnet run --project .\src\LearnDotnetCSharp.App -- list

# 某一分类
dotnet run --project .\src\LearnDotnetCSharp.App -- list networking

# 查看实验元数据
dotnet run --project .\src\LearnDotnetCSharp.App -- describe
networking.udp-websocket-tls

# 按 PDF 章节查找
dotnet run --project .\src\LearnDotnetCSharp.App -- list-chapter 16

# 单项、分类、全量
dotnet run --project .\src\LearnDotnetCSharp.App -- run memory.gc
dotnet run --project .\src\LearnDotnetCSharp.App -- run-category memory
dotnet run --project .\src\LearnDotnetCSharp.App -- run-all
```

建议日常使用 `run <id>`，每完成一个学习阶段再执行 `self-test`。`run-all` 适合浏览输出，但不适合第一次学习，因为连续输出会掩盖实验之间的概念边界。

2.1 四类结果与进程隔离

单项运行与全量回归回答的是两个不同问题：

- `run <id>` 在当前进程中运行，便于调试和连续观察输出；
- `self-test` 为每个 ID 启动全新的子进程，独立捕获标准输出和标准错误；
- 子进程退出码 0 表示 `Passed`，77 表示因平台能力缺失而 `Skipped`；
- 断言或未处理异常是 `Failed`；超过 15 秒会终止整个进程树并单独记录为 `Timeout`。

这种隔离不仅是测试技巧。线程池设置、静态缓存、`AssemblyLoadContext`、非托管崩溃、未释放句柄和忽略取消的死锁都是进程级状态；仅用同一进程中的 `CancellationToken` 无法强制恢复这些状态。显式结果分类也防止了另一种假绿：平台分支打印“跳过”后正常返回，外层却把它计作通过。

2.2 正式测试、故障计划与代码 CI

`tests/LearnDotnetCSharp.Tests` 使用 .NET 10 的 `Microsoft.Testing.Platform`，当前共 30 个正式测试。基础组验证目录数量、ID 唯一性、平台可用性、结果/退出码、输出截断和 `FaultPlan`；Capstones 组直接验证连续 checkpoint、死信幂等、源指纹拒绝、SQLite replay/conflict/single-flight/reopen、插件 v1/v2 路由和回退，以及 Python 常驻进程复用、崩溃替换、重复请求 ID 与取消清理。53 个端到端实验仍由进程隔离的 `self-test` 验证。二者不能互相替代：正式测试负责快速、精确定位状态转换，实验回归负责证明真实网络、文件、运行时和互操作边界仍能共同工作。

`FaultPlan` 用“命名故障点 + 第 N 次调用”描述可重复失败，并以原子计数保证并发调用中只触发一次。数据管线在 checkpoint 前中断，本地服务在 SQLite 已提交响应之后返回 503，最终 workflow 则先中断管线、再让 Python worker 按协议确定性退出。测试中的 `TaskCompletionSource` gate 与协议字段负责协调并发，不依赖随机数或用 `Sleep` 猜测时序。

`.github/workflows/build-and-test.yml` 在 Windows 与 Linux 上执行还原、Release 构建、格式检查、正式测试和 `self-test`。Windows 验证 MSVC C/C++ 路径；Linux 验证跨平台托管路径，并以可用性契约测试确认 Windows 专有实验显式 `Skipped`。仓库级离线还原便利设置不会在 CI 中隐藏 NuGet 源故障。

3. 五步学习闭环

每个实验至少完成以下五步：

1. **预测**：不运行代码，写下关键输出、线程/资源状态和可能的异常。
2. **运行**：记录实际结果，不只记录“通过”。
3. **解释**：用语言规范、BCL 契约和运行时机制解释差异。
4. **扰动**：只改变一个变量，例如容量、超时、比较器、协议版本或查询位置。
5. **复盘**：写出可迁移到真实项目的一条规则，以及实验不能证明的内容。

推荐实验记录模板：

```
## 实验： <ID>

- 日期与环境：
- 修改前预测：
- 实际输出：
- 与预测不一致之处：
- 源码中的关键边界：
- 我做的单变量修改：
- 新的结果与解释：
- 能迁移到生产代码的规则：
- 本实验不能证明什么：
```

4. 推荐进度

阶段	主题	建议时间	阶段产出
1	运行时、C# 14 与高级语言	12-16 小时	语言/运行时边界笔记
2	框架、集合、LINQ、XML/JSON	14-18 小时	查询与数据契约练习
3	异步、线程、并发、并行	16-22 小时	可取消并发管线
4	I/O、网络、内存与诊断	18-24 小时	本地协议服务与诊断报告
5	反射、编译器、正则与密码学	14-18 小时	插件/代码分析小项目
6	F#/VB/Python/C/C++ 互操作	16-24 小时	明确所有权的跨语言模块

时间不是考核指标。如果你不能在不看代码的情况下解释关键输出，就还没有完成该实验。

第二部分：运行时与高级 C# 语言

5. 运行时基线

5.1 runtime.overview

源码：[RuntimeOverviewDemo.cs](#)

运行：

```
dotnet run --project .\src\LearnDotnetCSharp.App -- run runtime.overview
```

先区分四个经常混淆的概念：

```
C# 14 源码
|
v
编译器 -> IL + 元数据 (面向 net10.0 的引用程序集)
```

```
|
v
.NET 10 运行时 -> 加载、验证、JIT/AOT、GC、线程与系统调用
|
v
操作系统与 CPU
```

重点观察：

- `FrameworkDescription` 与 `Environment.Version` 描述运行环境，不是语言版本；
- 进程架构决定原生库 ABI，不能只看操作系统架构；
- Workstation GC/Server GC 与延迟模式会影响吞吐、停顿和资源占用；
- `RuntimeFeature.IsDynamicCodeSupported/IsDynamicCodeCompiled` 决定反射发射、表达式编译等能力是否可用；
- NativeAOT 环境可能不允许本课程中的部分动态代码路径。

练习：

1. 分别在 Debug 与 Release 运行，说明哪些输出必然相同。
2. 查找 `TargetFramework`、`LangVersion` 和选中 SDK 的来源。
3. 写出“C# 14 功能可以在所有 .NET 10 部署模型中使用”为什么是错误命题。

5.2 runtime.jit-simd-pgo

源码：[JitSimdPgoDemo.cs](#)

运行：

```
dotnet run --project .\src\LearnDotnetCSharp.App -- run runtime.jit-simd-pgo
```

本实验把“结果正确”“机器具备能力”和“性能更快”拆成三个不同命题。标量循环与 `Vector<int>` 循环必须得到相同结果；`Vector.IsHardwareAccelerated`、`Vector128.IsHardwareAccelerated` 和 lane 数只描述当前进程的能力；X86 的 `Sse2` 与 Arm 的 `AdvSimd` 必须先检查 `IsSupported` 才能调用。没有硬件路径时，实验使用可移植后备实现，并保持同一结果契约。

分层编译和动态 PGO 也不能从一次调用直接下结论：

```
初始代码/ReadyToRun
-> Tier 0 快速 JIT + 可选插桩
-> 运行时收集热路径信息
-> Tier 1 优化代码（可能使用动态 PGO）
```

实验打印 `DOTNET_TieredCompilation`、`DOTNET_TieredPGO` 及旧 `COMPlus_` 覆盖项。`<unset>` 表示使用运行时默认值，不表示关闭；循环预热只制造调用历史，不能证明某个方法已在某一刻完成重编译。还要注意 `switch expression` 与其他运算符组合时应使用括号明确意图，例如 `(value % 5) switch`。

练习：

1. 在 x64 与 Arm64 各运行一次，比较能力和 lane 数，但不要预设哪台一定更快。
2. 分别设置 `DOTNET_TieredPGO=0/1` 运行 Release；只记录配置和正确性，不把本实验耗时作为基准。
3. 用 BenchmarkDotNet 新建独立基准，对比标量、`Vector<T>` 和硬件内建函数，并检查分配、离群值与反汇编。
4. 让数组长度不能整除 lane 数，解释尾部标量循环为何不可省略。

6. C# 14 正式语言特性

6.1 language.csharp14

源码: [CSharp14FeaturesDemo.cs](#)

本实验覆盖 extension members、**field**、null-conditional assignment、未绑定泛型 **nameof**、Span 转换、简化 lambda 参数修饰符、partial 构造函数/事件，以及用户定义复合赋值。

重点不是记语法，而是解释降级后的语义：

- extension member 仍是静态成员；调用形式不会改变其调度本质；
- **field** 指向编译器生成的后备字段，应注意同名成员遮蔽；
- **target?.Value = Expensive()** 在 **target** 为 **null** 时不会计算右侧；
- **nameof(List<>)** 只产生编译期字符串，不会构造泛型类型；
- Span 的更多隐式转换改善重载解析，但不会放宽 **ref struct** 的生命周期规则；
- partial 成员把声明与实现分开，适合生成器/手写代码协作；
- 用户定义 **+=** 可以原地更新，实验通过引用相等证明没有替换对象。

必须完成的修改：

1. 给 extension block 增加一个属性和一个运算符，检查是否产生实例成员。
2. 在 null-conditional assignment 右侧增加计数器，分别测试 null/非 null。
3. 打开编译器生成文件或反编译程序集，定位 partial event 的最终成员。

6.2 language.advanced

源码: [AdvancedLanguageDemo.cs](#)

概念主线：

- 静态抽象接口成员让运算能力进入泛型约束，**Mean<T>** 不必依赖 **dynamic**；
- record 层次提供值语义和模式匹配友好形状，但 C# 类层次并不是原生判别联合；
- 属性模式、关系模式、列表模式表达数据形状，不应把副作用藏进模式访问器；
- **ref struct** 和 **Span<T>** 受逃逸分析约束，不能随意进入堆对象、闭包和异步状态机；
- **stackalloc**/指针绕过部分安全保护，只应存在于小而可审计的边界；
- **dynamic** 把部分检查推迟到运行时，失败会变成绑定异常。

练习：

1. 为 **Mean<T>** 增加 **Half** 或自定义数值类型，记录约束是否足够。
2. 给 record 层次增加新派生类型，检查现有 switch 是否被编译器强制更新。
3. 尝试让 Span 跨越 **await**，阅读编译器错误并解释生命周期原因。
4. 把动态调用改为泛型或接口调用，比较错误发生的阶段。

6.3 language.operators-conversions

源码: [OperatorsAndConversionsDemo.cs](#)

本实验用缩放因子为 100 的 **FixedPoint** 值类型集中展示传统运算符和转换运算符。选择定点数而不是业务 **Money**，是为了把舍入、溢出和类型转换语义与币种、税率等领域规则分开。

必须掌握的规则：

- 只能重载语言允许的既有运算符，不能发明新符号；比较与相等运算符要按规定成对定义，并让 `Equals/GetHashCode` 与 `==` 保持一致；
- 隐式转换应当无损、稳定且不抛异常；可能丢失精度、改变单位或失败的转换应设计为显式；
- 普通与 `checked` 用户定义运算符可以并存，调用点的 `checked` context 决定选择 `op_Addition` 还是 `op_CheckedAddition`、`op_Explicit` 还是 `op_CheckedExplicit`；
- 非 nullable 值类型运算符会自动提升到 nullable：任一操作数没有值时，提升后的结果通常也没有值；
- `is` 和 `as` 检查运行时类型关系，不考虑用户定义转换；显式 `cast` 才能触发转换运算符；
- `Convert.ChangeType` 依赖 `IConvertible` 等转换协议，不会因为类型声明了 `implicit operator` 就自动发现它；
- 用户定义转换让调用代码更短，却也会扩大重载解析表面；优先保证语义自然，而不是追求“任何类型都能互转”。

实验中的关键对照：

```
int --implicit--> FixedPoint --implicit--> decimal
decimal --explicit--> FixedPoint --explicit/checked--> int

FixedPoint? + FixedPoint? -> FixedPoint?
object is/as FixedPoint -> 不调用用户定义转换
```

练习：

1. 删除 `checked` 加法或 `checked` 显式转换，观察 `checked` context 如何回退以及输出如何改变。
2. 把舍入策略从 `ToEven` 改为 `AwayFromZero`，为正负中点值分别增加断言。
3. 实现 `IAdditionOperators<TSelf, TOther, TResult>`，让 `FixedPoint` 进入一个泛型求和算法。
4. 用 IL 反汇编定位 `op_Implicit`、`op_Explicit`、`op_CheckedExplicit`、`op_Addition` 和 `op_CheckedAddition`。
5. 尝试设计带单位的长度类型，说明哪些转换可以隐式，哪些必须显式，哪些根本不应存在。

6.4 language.generics-variance-constraints

源码：[GenericsVarianceConstraintsDemo.cs](#)

这个实验集中回答“泛型参数到底能做什么”。约束不是运行时筛选器，而是编译期能力声明：编译器据此允许成员调用、栈上布局、构造和静态分派，并在调用点拒绝不满足条件的类型。

关键机制：

- `IProducer<out T>` 只在输出位置使用 `T`，允许生产者协变；`IConsumer<in T>` 只在输入位置使用 `T`，允许消费者逆变；型变只适用于引用类型构造。
- `notnull` 表达 nullable 契约，`unmanaged` 允许安全地推断没有托管引用并使用 `sizeof(T)`，`new()` 要求可访问无参构造器且必须放在约束列表末尾。
- 静态抽象接口成员让 `T.Combine(...)` 进入泛型算法，不需要 `dynamic` 或反射；实现由具体类型在编译期约束下提供。
- `where T : allows ref struct` 是反约束：泛型代码承诺能够接收 byref-like 类型，并必须遵守其逃逸规则。实验用它保存 `Span<int>`，但不会让 `Span` 获得进入堆或跨 `await` 的权限。
- 类型推断来自实参、目标类型与约束的共同作用；约束通常不单独作为推断来源。

练习：

1. 把协变接口中的 `T` 用作方法参数，阅读编译器错误并解释“不安全写入”。
2. 尝试用 `int` 做型变转换，说明为什么值类型构造不参与引用型变。

3. 为静态抽象接口增加 **Zero**，把 **Fold** 改成空序列也有定义的聚合。
4. 给 **unmanaged** 方法传入含 **string** 字段的结构体，记录错误发生在编译期而不是运行期。
5. 尝试把 **RefSlot<Span<int>>** 捕获到 **lambda** 或带过 **await**，解释反约束没有取消生命周期检查。

6.5 language.attributes

源码：[AttributesAdvancedDemo.cs](#)

Attribute 是写入元数据的声明性信息，不是自动执行的拦截器。要产生行为，必须有编译器、运行时、框架、源生成器或你的反射代码读取它。

按顺序理解：

1. **AttributeUsage** 决定目标、是否允许多实例、是否继承；
2. 构造参数进入固定参数，属性设置进入命名参数；
3. **CustomAttributeData** 可在不实例化 Attribute 的情况下查看原始元数据；
4. 参数 Attribute 可以驱动验证约定，但验证器必须显式实现；
5. **CallerMemberName** 等由编译器在调用点填值；
6. **Conditional** 可以让调用本身在编译期消失，而不是在运行时做一次 if。

练习：实现一个 **[Range]** 风格的参数契约，要求验证失败时禁止反射调用。再比较运行时反射验证与源生成验证的优缺点。

6.6 language.delegates-events-iterators

源码：[DelegatesEventsIteratorsDemo.cs](#)

重点观察：

- 闭包捕获变量的位置，而不是创建委托那一刻的值；
- **static** lambda 禁止捕获，有助于显式表达无环境依赖；
- 事件限制外部调用，只允许订阅/退订；发布者仍需考虑订阅者生命周期；
- exception filter 在进入 catch 前判断，适合按错误码筛选；
- **Flags** 枚举值应使用独立位；
- **yield** 把方法改写为状态机，枚举发生时才执行；
- 匿名类型适合局部投影，命名元组适合轻量返回值，但都不是稳定的跨进程契约。

练习：

1. 在循环中创建闭包并预测每个委托读取的值。
2. 故意不退订长生命周期发布者的事件，画出可能的对象保留链。
3. 在 Fibonacci 迭代器中加入日志，证明创建序列不等于开始执行。

第三部分：框架基础、集合与数据查询

7. 文本、时间、数值和标识

7.1 `framework.text-globalization`

源码： [TextAndGlobalizationDemo.cs](#)

同一段文本至少有四种不同长度：UTF-16 code unit、Unicode scalar value、用户感知文本元素、编码后的字节数。`string.Length` 只回答第一种。

重点规则：

- 网络/文件协议先确定编码，通常显式使用 UTF-8；
- 标识符、协议字段常用 ordinal 比较；面向人的排序/搜索才考虑文化规则；
- 解析外部持久化数据时显式指定固定区域性；
- UI 格式化可以使用用户区域性，但不要把 UI 字符串反过来当协议。

练习：加入 emoji、组合附加符号和德语字符，比较四种计数；再用 Turkish culture 测试大小写比较。

7.2 `framework.time-numeric-identity`

源码： [TimeNumericsAndIdentityDemo.cs](#)

重点区分：

- `DateTimeOffset` 适合表示时间线上的时间点；
- `TimeZoneInfo` 负责时区规则转换；
- `DateOnly/TimeOnly` 表达没有时区含义的日历值；
- `BigInteger` 用于超出固定整数范围的精确整数；
- 舍入必须明确 `MidpointRounding` 策略；
- `BitOperations` 表达位级算法；
- `Flags` 表达位集合；
- `UUID v7` 具有时间有序特征，但仍不能替代数据库业务排序字段或授权逻辑。

练习：构造夏令时无效/歧义时间，验证“当地时间字符串 + 时区”为什么比单独 `DateTime` 更复杂。

8. 集合与自定义集合契约

8.1 `collections.core-custom-buffer`

源码： [CollectionsAndCustomBufferDemo.cs](#)

不要先问“哪个集合最快”，先问需要什么语义：

需求	典型选择
先进先出	<code>Queue<T></code>
后进先出	<code>Stack<T></code>
按优先级取出	<code>PriorityQueue<TElement, TPriority></code>
唯一性与集合运算	<code>HashSet<T></code>

需求	典型选择
排序后的唯一集合	<code>SortedSet<T></code>
构建后高频只读查找	<code>FrozenDictionary<TKey, TValue></code>
固定容量、覆盖最旧值	自定义 ring buffer

自定义 `ICollection<T>` 必须保证 `Count`、索引器和枚举的逻辑顺序一致。环形缓冲区的物理数组顺序不等于使用者看到的逻辑顺序。

练习：

1. 给 ring buffer 增加 `TryGetNewest`，不破坏现有契约。
2. 替换字符串比较器，观察唯一性和查找结果变化。
3. 为索引越界、容量 1、覆盖多轮增加断言。

9. LINQ：从委托到真实 Provider

9.1 `linq.query-semantics`

源码：[LinqQuerySemanticsDemo.cs](#)

学习主线：

```
IEnumerable<T> -> 委托 + 本地对象 + 枚举时执行
IQueryable<T> -> Expression + Provider + 翻译/执行边界
```

需要解释：

- 延迟查询在每次枚举时重新读取当前数据源；
- `ToArray/ToList` 建立物化快照；
- query syntax 最终会翻译成标准查询运算符；
- `let` 可保存中间表达式；
- join 与相关子查询有不同形状和成本；
- `AsQueryable()` 用在普通 List 上仍是 LINQ to Objects provider，不会凭空产生 SQL。

练习：对同一延迟查询枚举两次，在中间修改数据源，记录谓词调用次数。

9.2 `linq.operators`

源码：[LinqOperatorsDemo.cs](#)

按运算符家族学习，而不是逐个背诵：

- 筛选/投影：`Where`、`Select`、`SelectMany`；
- 分区：`Take`、`Skip`、范围与按条件分区；
- 集合：`Distinct`、`Union`、`Intersect`、`Except`；
- 连接：`Join`、`GroupJoin`、`.NET 10 LeftJoin/RightJoin`；
- 分组/聚合：`GroupBy`、`CountBy`、`AggregateBy`、`Aggregate`；
- 元素/量词：`First`、`Single`、`Any`、`All`；
- 顺序与物化：`OrderBy`、`ThenBy`、`ToLookup`、`ToDictionary`。

重点分析空序列、重复键、稳定顺序、比较器和多次枚举。`Single` 表达“必须恰好一个”的业务不变量，不是 `First` 的更严格拼写。

9.3 `linq.efcore-sqlite-provider`

源码: [EfCoreSqliteProviderDemo.cs](#)

这是理解 `IQueryable<T>` 的关键实验。它使用进程内 SQLite, 让 EF Core 10 把表达式树翻译成参数化 SQL。

观察顺序:

1. 查看 `providerQuery.Expression`;
2. 调用 `ToQueryString()`, 确认出现参数、`LEFT JOIN` 与 `ORDER BY`;
3. 到 `ToArrayAsync` 才真正异步执行并物化;
4. `AsNoTracking` 表示结果不进入变更跟踪;
5. 先物化, 再用 `AsEnumerable` 执行只属于本地代码的格式化方法。

关键边界: 不要用 `AsEnumerable` 掩盖本应在数据库执行却无法翻译的筛选。它可能把大量数据拉到客户端, 并改变性能和一致性语义。

练习:

1. 把 `Where` 移到 `AsEnumerable` 后, 比较 SQL 和数据处理位置。
2. 加入 provider 无法翻译的方法, 记录异常发生在构造查询还是执行查询。
3. 去掉 `AsNoTracking`, 检查 `ChangeTracker.Entries()`。
4. 为左连接增加没有匹配项的分类, 验证 null 投影。

10. XML 与版本化序列化

10.1 `xml.linq`

源码: [LinqToXmlDemo.cs](#)

LINQ to XML 把 XML 看作可查询、可修改的对象树。重点学习:

- 函数式构造;
- `XNamespace` 与限定名;
- 元素/属性查询和类型转换;
- `SetElementValue`、增加/删除节点;
- `XObject` annotation 保存不进入 XML 文本的对象侧信息。

练习: 加入默认命名空间与前缀命名空间, 故意省略 `XNamespace`, 解释为什么查询结果为空而不是抛异常。

10.2 `serialization.xml-json`

源码: [XmlJsonStreamingDemo.cs](#)

对比两种处理模型:

对象树/DOM: 易于随机访问和修改, 内存通常随文档增长
流式 Reader/Writer: 单遍、低驻留内存, 需要显式维护读取状态

实验使用异步 `XmlWriter/XmlReader` 和 `System.Text.Json` 源生成。重点观察严格输入设置、异步流生命周期, 以及源生成上下文如何减少运行时反射依赖。

10.3 `serialization.contract-versioning`

源码: [ContractVersioningDemo.cs](#)

序列化首先是协议设计，其次才是调用 API。必须显式决定：

- 根元素、元素/属性名和 XML namespace；
- JSON 属性名与命名策略；
- 允许哪些派生类型；
- 多态判别器如何命名；
- 遇到未知字段时忽略、保存还是拒绝；
- 旧读者和新读者的兼容策略；
- 输入大小、深度、重复属性和 DTD 等安全限制。

实验通过 `JsonExtensionData` 保留未来字段，并用白名单判别器恢复多态运行时类型。它明确不恢复 `BinaryFormatter`。

练习：

1. 模拟 v3 写入、v2 读取并重新写出，检查未知字段是否保留。
2. 添加未注册的派生类型，确认失败策略。
3. 给 XML 输入加入 DTD，验证安全 reader 拒绝它。

第四部分：异步、线程与并发

11. 异步控制流

11.1 `async.cancellation-stream-valuetask`

源码：[AsyncCancellationStreamDemo.cs](#)

建立以下模型：

- `async` 表示可暂停的控制流，不等于创建线程；
- 取消是合作协议，`CancellationToken` 不会强行终止任意代码；
- 超时通常由取消构建，但业务取消与超时应在错误语义上区分；
- `IAsyncEnumerable<T>` 在每次 `MoveNextAsync` 之间允许异步等待；
- `ValueTask<T>` 适合同步完成概率高且性能证据充分的路径，不应默认替代 `Task<T>`；
- 除非已转换为 `Task` 或明确知道来源允许，否则不要多次 `await` 同一个 `ValueTask`。

实验要点：首次查找需要异步工作，缓存命中可以同步完成；异步流在消费若干元素后观察取消；独立超时路径证明超时不是“任务自动消失”。

练习：把取消检查从生产循环移除，观察消费者取消后的资源生命周期。

11.2 `async.task-when-each`

源码：[TaskWhenEachDemo.cs](#)

`Task.WhenAll` 的结果按输入任务顺序排列；`Task.WhenEach` 让你按完成顺序逐个观察任务。两者表达不同需求：

- 全部完成后统一处理：`WhenAll`；
- 尽快处理已完成项、刷新进度或流式输出：`WhenEach`。

练习：加入一个失败任务和一个取消任务，记录异常在哪一步被观察。不要写依赖具体调度时间的脆弱断言。

11.3 `async.continuations-context`

源码: [ContinuationsAndContextDemo.cs](#)

实验创建一个带消息泵的专用单线程 `SynchronizationContext`, 因此可以重复观察上下文捕获, 而不依赖 UI 框架。先建立两个互相独立的模型:

机制	回答的问题	典型内容
<code>SynchronizationContext</code>	continuation 应在哪里运行	UI/专用事件线程的调度目标
<code>ExecutionContext</code>	continuation 应看到哪些 ambient 状态	<code>AsyncLocal<T></code> 、安全上下文等

普通 `await Task.Yield()` 会把 continuation 投递回专用上下文; `ConfigureAwait(false)` 不恢复该调度目标, 但不会自动阻止 `ExecutionContext` 和 `AsyncLocal<T>` 流动。`ExecutionContext.SuppressFlow()` 只影响在抑制范围内新排队的工作, 不能把它当成每个 `await` 的优化开关。

同步等待实验故意在单线程上下文内调用 `Wait(75 ms)`: 被等待任务的 continuation 也想回到同一线程, 线程却正在等待它, 于是先出现可控超时。超时后消息泵恢复, 任务最终得到 42。这个设计只呈现死锁结构, 不留下永久挂起线程; 生产代码应一路 `await`, 而不是依赖超时“修复” sync-over-async。

练习:

1. 删除 `ConfigureAwait(false)`, 比较 continuation 的线程和当前上下文。
2. 把有限 `Wait` 改成 `.Result` 前先画等待图; 不要实际把永久死锁提交到主线。
3. 分别在 `Task.Run` 创建前后调用 `SuppressFlow`, 解释作用域为什么重要。
4. 让异步方法抛出嵌套异常, 比较 `await` 与 `AggregateException` 风格同步观察的调用栈。

12. 线程、执行上下文与同步

12.1 `threading.execution-context`

源码: [ThreadExecutionContextDemo.cs](#)

区分:

- 专用 `Thread`: 显式创建的 OS 线程;
- `ThreadPool`: 复用工作线程执行短任务;
- `ExecutionContext`: 随逻辑执行流传播的环境;
- `AsyncLocal<T>`: 逻辑调用上下文, 不等于线程本地存储;
- `SuppressFlow`: 阻止环境进入新排队的工作, 不应被滥用为“性能优化”。

实验分别观察专用线程、正常 `ThreadPool` 排队和抑制传播后的 ambient value。

12.2 `threading.synchronization-primitives`

源码: [SynchronizationPrimitivesDemo.cs](#)

按问题选择原语:

问题	可考虑的原语
限制并发度	<code>SemaphoreSlim</code>
一次性打开闸门	<code>ManualResetEventSlim</code>
等待 N 个参与者完成	<code>CountdownEvent</code>

问题	可考虑的原语
简单数值原子更新	<code>Interlocked</code>
保护复合不变量	<code>lock</code> /其他互斥机制

原子不等于无竞争，线程安全也不等于公平。`Interlocked.CompareExchange` 循环适合单个值的读-改-写，不适合跨多个字段自动维持业务不变量。

练习：把最大值更新改成非原子读写并重复运行，解释为什么偶尔“看起来正确”不能证明线程安全。

13. 消息传递与数据并行

13.1 `concurrency.channel-pipeline`

源码：[ChannelPipelineDemo.cs](#)

有界 `Channel<T>` 同时表达：

- 生产者与消费者解耦；
- 容量上限；
- 写满后的背压；
- 完成与失败传播；
- 多生产者/消费者的消息所有权。

实验使用并发状态和 `Interlocked` 验证每项恰好处理一次、总数和总和正确。学习时重点检查 `writer` 是否总能 `Complete`，消费者是否处理 `completion` 异常，以及取消时谁负责关闭管线。

13.2 `concurrency.parallel-plinq`

源码：[ParallelPlinqDemo.cs](#)

`Parallel.For` 与 PLINQ 面向 CPU 并行，不是异步 I/O 的替代品。

要分析：

- 工作量是否足以覆盖调度和分区成本；
- 操作是否独立；
- 是否要求输出顺序；
- 聚合是否线程安全；
- 取消和多个异常如何传播；
- 测试结果是否在 Release、无调试器、预热后获得。

练习：比较 `AsOrdered` 前后顺序和成本；把工作量降低，观察并行版本不一定更快。

13.3 `concurrency.memory-model-lock-free`

源码：[MemoryModelAndLockFreeDemo.cs](#)

这个实验从“共享内存怎样被正确发布”推进到 CAS 数据结构。`Volatile.Write` 发布 `ready` 标志，`Volatile.Read` 获取它；消费者看到标志后再读取此前写入的 `payload`。`volatile` 不会把多个字段组合成事务，也不能替代复杂不变量的互斥保护。

`Interlocked.CompareExchange` 返回目标位置的旧值，因此典型更新必须是重试循环：读取快照、计算候选值、尝试替换；若其他线程先修改，就用新快照重算。实验以同一模式实现 CAS counter 和 Treiber stack，并验证并

发 push 后元素总数和唯一性。

ABA 对照刻意把状态从 A 改成 B 再改回 A：只比较值的 CAS 无法知道中间发生过变化；把值和版本号打包进同一个原子字后，版本变化可以拒绝陈旧比较。版本戳仍需考虑溢出、回收和节点生命周期，生产代码通常优先使用经过验证的并发集合。

最后的显式布局把两个计数器放到偏移 0 和 64，只证明字段距离，不证明当前 CPU 的缓存行大小，也不证明性能改善。伪共享必须在目标硬件、Release 构建和专业 profiler/benchmark 中验证。

练习：

1. 把安全发布改成普通 flag，说明即使多次运行正确也不能提升为内存模型保证。
2. 记录 CAS retries 随并发度变化的趋势，但不要对具体次数做断言。
3. 画出 Treiber push 的线性化点，并说明节点何时对其他线程可见。
4. 扩展版本戳 ABA 示例，讨论计数器回绕和节点复用问题。
5. 用 `ConcurrentStack<T>` 替换自制实现，比较 API 契约、维护成本和可观测行为。

第五部分：I/O 与网络协议

14. 流、管道与随机访问

14.1 `io.streams-pipelines-compression`

源码：[StreamsPipelinesCompressionDemo.cs](#)

核心问题不是“怎样调用 Read”，而是：数据边界在哪里、缓冲区归谁、读到一部分怎么办。

- `Stream.ReadAsync` 允许短读；
- GZip 是变换流，释放顺序会影响尾部是否写完；
- `PipeReader` 可能返回多段 `ReadOnlySequence<byte>`；
- `AdvanceTo(consumed, examined)` 告诉管道哪些数据已消费、哪些已检查；
- 不要为了方便把所有输入无上限复制到一个数组。

练习：把一条记录拆在两个缓冲段中，确认解析器仍然正确；再故意错误设置 `AdvanceTo`，观察重复读取或停滞。

14.2 `io.random-access-memory-map`

源码：[RandomAccessMemoryMapDemo.cs](#)

`RandomAccess` 以文件句柄和显式偏移工作，没有共享 `Position`，适合并发处理独立区间。内存映射把文件区域映射为进程地址空间，适合随机访问和共享视图，但仍需处理：

- 映射/视图生命周期；
- 刷新与可见性；
- 文件大小和偏移边界；
- 其他进程修改；
- 不能把映射地址保存在生命周期更长的对象中。

练习：用两个并发任务写不同偏移，再用映射视图验证；随后故意让区间重叠并定义冲突策略。

15. TCP、HTTP、UDP、WebSocket 与 TLS

先记住抽象层次：

```
应用消息/契约
|-- HTTP/1.1, HTTP/2, HTTP/3
|-- WebSocket
|-- 自定义帧协议
传输与安全
|-- TLS over TCP
|-- QUIC (内含 TLS 1.3 机制)
|-- TCP
`-- UDP
IP / loopback / 网络接口
```

15.1 networking.tcp-loopback

源码: [TcpLoopbackDemo.cs](#)

TCP 是可靠、有序的字节流，但没有消息边界。一次 **Write** 不保证对应一次 **Read**。实验必须使用长度或分隔符等 framing 规则，并在异常路径释放 listener、client 和 stream。

练习：把一条消息拆成多次写入，确认接收端仍按协议重组，而不是依赖本次机器的分包巧合。

15.2 networking.httpclient-loopback

源码: [HttpClientLoopbackDemo.cs](#)

关注 **HttpClient** 生命周期、请求超时、取消、状态码、内容读取和服务端资源释放。真实应用通常复用 client/handler 或使用 factory，不要为每个请求创建一个长期产生连接压力的新 handler。

15.3 networking.http2-http3

源码: [Http2And3LoopbackDemo.cs](#)

实验在本地 Kestrel 上创建临时证书，以 SHA-256 指纹只信任本次证书，并用 **RequestVersionExact** 验证真实协商结果。

- HTTP/2: TCP + TLS/ALPN;
- HTTP/3: QUIC，并依赖平台 MsQuic/QUIC 支持;
- 请求版本只是意图，响应版本才是结果;
- 条件式 **SKIPPED** 表示平台能力缺失，不应伪装为 HTTP/3 成功。

练习：把版本策略改为允许降级，对比为什么测试可能“通过”却没有验证目标协议。

15.4 networking.udp-websocket-tls

源码: [UdpWebSocketTlsDemo.cs](#)

这个实验把四种常被混在一起的能力拆开：

- DNS：名称到地址的解析；
- UDP：保留数据报边界，但不承诺到达、顺序和去重；
- WebSocket：经 HTTP Upgrade 建立的双向消息/帧协议；
- **SslStream**：在 TCP 字节流上提供身份验证、机密性和完整性。

TLS 部分验证自签名证书的精确 SHA-256 指纹，协商 TLS 版本和自定义 ALPN，再交换 **PING/PONG**。生产环境通常应使用系统信任链或受控 PKI，而不是复制“信任任意自签名证书”的回调。

练习：

1. 修改 ALPN 使两端不匹配，记录握手行为。

2. 修改证书验证回调使指纹不匹配，确认业务数据不会发送。
3. 为 UDP 加入消息 ID，设计去重与超时重试；明确它仍不等于 TCP。

15.5 networking.http-streaming-resilience

源码：[HttpStreamingResilienceDemo.cs](#)

本实验覆盖三个容易在简单 HTTP 示例中遗漏的生产边界：

1. `ResponseHeadersRead` 让 `SendAsync` 在响应头可用后返回，正文需要调用方继续读取、设置取消并负责释放 response；实验用 `Pipe` 驱动的分段 `HttpContent` 消除操作系统小包缓冲的偶然性，再用 `DeserializeAsyncEnumerable<T>` 增量消费 JSON 数组。
2. 回环 Kestrel 的 `/flaky` 第一次返回 503、第二次成功。重试方法每次创建新请求，并且只为本例明确允许的幂等 GET 和瞬时状态码重试。
3. `/slow` 持续刷新响应，客户端取消正文读取后，服务器通过 `HttpContext.RequestAborted` 观察连接终止。取消既要传给发送，也要传给后续正文读取。

这里故意把“可控内容流”和“真实 loopback 传输”分开：前者稳定证明客户端完成选项，后者证明状态码、连接与服务器取消链路。它没有实现完整韧性库。生产策略还需要退避、抖动、Retry-After、总时间预算、熔断、可观测性和请求体重放规则；POST 等操作通常还需要幂等键或业务去重。

练习：

1. 改用默认 `ResponseContentRead`，在不打开 gate 的情况下预测为什么调用不能完成。
2. 给流内容设置独立于 `HttpClient.Timeout` 的短取消，验证头已收到后仍能终止正文。
3. 为 429 读取 `Retry-After`，并给总重试时间设置上限。
4. 尝试重试带一次性 `StreamContent` 的 POST，解释为什么“重新 new request”仍不够。
5. 在 Activity 中记录 attempt、status code 和最终结果，但避免把请求体写入 tag。

第六部分：内存、GC 与资源生命周期

16. GC、Span 与所有权

16.1 memory.gc

源码：[GarbageCollectionDemo.cs](#)

GC 管理托管内存，不负责及时释放所有外部资源。重点观察：

- 分代基于“多数对象存活很短”的经验；
- 对象是否回收取决于可达性，不取决于变量是否离开肉眼可见的代码块；
- 弱引用不阻止回收；
- `GC.GetAllocatedBytesForCurrentThread` 与堆总体信息回答不同问题；
- `ArrayPool<T>` 降低重复数组分配，但租出的数组可能更大且含旧数据；
- 手动 `GC.Collect` 主要用于受控教学/诊断，不是常规性能方案。

练习：改变对象大小和存活时间，多次运行并区分语义保证与本次 GC 时机。

16.2 memory.span-memory-ownership

源码：[SpanMemoryOwnershipDemo.cs](#)

`Span<T>` 是受限生命周期的同步视图; `Memory<T>` 可以存入对象并跨 `await`。它们都不自动拥有底层数据。

缓冲区所有权问题必须回答:

1. 谁分配/租用?
2. 谁能读写?
3. 有效期到什么时候?
4. 谁归还/释放?
5. 归还后是否还有别名?
6. 是否需要清除敏感内容?

练习: 故意在归还池后保留 `Memory<T>`, 说明即使本次输出正确, 代码仍然违反所有权契约。

16.3 `memory.disposal-finalization`

源码: [DisposalAndFinalizationDemo.cs](#)

推荐层次:

原生句柄 -> `SafeHandle` 子类
拥有 `SafeHandle`/托管资源的类型 -> `IDisposable` / `IAsyncDisposable`
调用方 -> `using` / `await using`

`SafeHandle` 把关键原生释放放在可靠抽象中, 拥有者通常不再需要自己的 `finalizer`。**`Dispose`** 提供确定性释放; `finalizer` 只是一条不确定、昂贵的最后防线。成功 `Dispose` 后用 `GC.SuppressFinalize` 避免不必要终结。

实验用单独 `probe` 观察 `Dispose` 路径和 `finalizer` 路径, 不代表生产代码应依赖 `GC.Collect` 驱动终结。

练习:

1. 连续调用两次 `Dispose`, 验证幂等。
2. 模拟异步 `flush`, 说明为什么需要 `IAsyncDisposable`。
3. 画出拥有者、`SafeHandle`、原生内存之间的生命周期。

16.4 `memory.pinning-native-memory`

源码: [PinningAndNativeMemoryDemo.cs](#)

固定对象的目的, 是在一段明确生命周期内阻止 GC 移动对象, 从而向原生代码提供稳定地址。实验对比两种方式:

- `GC.AllocateUninitializedArray(..., pinned: true)` 直接创建 pinned heap 数组;
- `GCHandle.Alloc(array, GCHandleType.Pinned)` 临时固定普通数组, 并在 `finally` 中 `Free`。

两者都在受控压缩 GC 前后比较地址, 但实验只断言“固定期间稳定”, 不声称普通数组一定会移动。长期或大量固定会限制压缩并可能增加碎片, 因此固定范围应尽可能短。

`BinaryPrimitives` 显式指定 little/big endian; `MemoryMarshal.Cast<byte, int>` 则按本机端序解释同一批字节。两者的输出对照说明“内存视图”不等于“协议编码”。跨机器协议必须指定端序, 而不能依赖 `BitConverter.IsLittleEndian` 的当前值。

原生部分用 `NativeMemory.AlignedAlloc(256, 64)` 验证地址对齐, 并始终用匹配的 `AlignedFree` 释放; 另一个缓冲区由 `SafeHandleZeroOrMinusOneIsInvalid` 子类拥有, `ReleaseHandle` 调用 `NativeMemory.Free`。指针、`Span` 与 `handle` 都不能在释放后继续使用。

练习:

1. 把 GCHandle 的 **Free** 移出 finally，列出异常路径上的后果，再恢复正确代码。
2. 给原生缓冲区增加长度边界检查和 **ReadOnlySpan<byte>** 只读视图。
3. 故意交换 little/big endian 读取，写出一个不会依赖当前 CPU 端序的协议测试。
4. 用 profiler 观察大量短固定和少量长期固定的差异；不要从地址输出推导碎片程度。
5. 为 SafeHandle 所有者增加幂等 Dispose 测试，并解释为什么调用方不应直接缓存 **DangerousGetHandle()**。

第七部分：诊断、反射、程序集与编译器

17. 可观测性与诊断

17.1 **diagnostics.observability**

源码：[ObservabilityDiagnosticsDemo.cs](#)

不同信号回答不同问题：

信号	主要问题
日志/TraceSource	发生了什么
Activity/trace	一次操作经过了哪些组件
Meter/metric	系统整体趋势和分布怎样
EventSource	低层、高吞吐运行时/组件事件
StackTrace	当前调用路径是什么
Stopwatch	一段代码经历了多少时间

实验验证 **Activity.Current** 跨 await 传播、Meter listener 收到 counter/histogram、EventListener 收到强类型事件，以及栈捕获与计时。

生产设计问题：

- tag 是否包含高基数或敏感数据；
- 采样在哪里发生；
- 没有 listener 时是否避免昂贵 payload；
- 指标单位和直方图边界是否稳定；
- trace/log 是否能通过 trace ID 关联。

练习：为一个异步子操作创建 child Activity，验证父子 ID；再添加高基数 tag，说明代价。

18. 反射、插件加载与编译器

18.1 **reflection.advanced**

源码：[ReflectionAdvancedDemo.cs](#)

实验依次展示 Attribute 元数据、开放/封闭泛型方法、反射调用、表达式树和 **Reflection.Emit**。

需要区分：

- 反射读取元数据；
- **MethodInfo.Invoke** 进行运行时调用；
- expression tree 是可检查的数据结构；

- `Compile()` 把表达式变成可执行委托;
- `Reflection.Emit` 直接构造 IL;
- 动态代码在 NativeAOT 等环境可能不可用。

练习：在关闭泛型方法前传入不满足约束的类型，记录错误阶段；再为 `Emit` 方法加入 `checked overflow` 测试。

18.2 runtime.collectible-plugin

源码：CollectiblePluginDemo.cs

配套项目：

- 插件契约
- 示例插件
- 日语资源

插件模型：

```
Default AssemblyLoadContext
`-- PluginContract (宿主与插件共享类型身份)

Collectible PluginLoadContext
|-- SamplePlugin
`-- ja-JP satellite resource
```

如果契约程序集在两个上下文各加载一份，即使全名相同，类型身份也可能不兼容。

`AssemblyDependencyResolver` 帮助定位依赖；`Unload()` 只是发起卸载，真正回收要等线程、静态字段、委托、反射对象和实例等所有根消失。

练习：故意把插件实例存入宿主静态字段，观察 `weak reference` 为什么仍存活；清除引用后再验证卸载。

18.3 compiler.roslyn-il

源码：CompilerAndIldemo.cs 与 ILDisassembler.cs

完整管线：

```
源码文本
-> SyntaxTree (语法结构)
-> SemanticModel (符号、类型、绑定)
-> Compilation (引用 + 诊断)
-> Emit (PE/PDB)
-> AssemblyLoadContext (加载)
-> MethodBody IL (反汇编)
-> JIT/AOT (本机代码)
```

实验既编译内存源码，也比较普通方法与 `async` 方法。`async` 方法通常生成状态机；入口方法不再包含你在源码中看到的完整顺序控制流。

引用边界：通用编译工具应使用目标框架 `reference assemblies`。只引用当前运行时 `core library` 的极简方式仅适用于受控实验。

练习：

1. 给内存源码增加语义错误，打印诊断位置和 ID。
2. 编译一个 `iterator`，寻找生成的状态机类型和 `MoveNext`。
3. 比较 `Debug/Release IL`，解释哪些差异来自优化。

18.4 compiler.incremental-generator-analyzer

源码: [IncrementalGeneratorAnalyzerDemo.cs](#)

实验在内存中构建一份 C# 14 Compilation, 并真实运行两种 Roslyn 扩展:

用户语法树

```
|-- IncrementalGenerator: 筛选 partial Greeter -> 新增 .g.cs  
`-- DiagnosticAnalyzer: 语法候选 -> SemanticModel 确认 Task<TResult>.Result -> LDCS001
```

原始语法树 + 生成语法树 -> Emit -> 可加载程序集

增量生成器使用 syntax provider, 只把匹配节点转换成稳定的小值, 再在输出阶段生成 partial 类型。生成器实例生命周期由编译器控制, 不应把跨轮次正确性建立在实例可变量段上。实验检查生成树数量、生成文件名、零错误诊断, 并加载发射出的程序集读取 **Greeting** 属性, 证明生成代码真正进入编译产物。

分析器先做廉价语法筛选, 再通过语义模型确认访问的是 **System.Threading.Tasks.Task<TResult>.Result**, 避免把任何名为 Result 的属性都误报。它只报告 **LDCS001**, 不修改源码; 代码修复应作为独立、用户可选择的操作。真实分析器还需要并发安全、生成代码策略、诊断位置、可配置严重级别、误报控制和测试矩阵。

实验通过 **TRUSTED_PLATFORM_ASSEMBLIES** 取得当前运行环境的引用, 以便内存样例独立编译。面向其他 TFM 的通用工具应使用对应 reference assemblies, 不能把当前运行时程序集错误当成任意目标框架的编译契约。

练习:

1. 增加第二个 partial 类型, 决定 hint name 如何避免冲突并保持确定性。
2. 给生成器加入 marker Attribute; 比较只看语法名称与语义确认 Attribute 的误匹配风险。
3. 让分析器识别 **.Wait()**, 但排除已完成任务或测试代码之前先定义可证明的规则。
4. 为 LDCS001 增加 code fix 草案: 把同步成员访问改成 await 需要哪些调用链变更?
5. 把输入源码加入编译错误, 分别记录生成器诊断、编译诊断和分析器诊断的来源。

第八部分: 正则表达式与密码学

19. 正则表达式的复杂度边界

19.1 regex.advanced

源码: [RegexAdvancedDemo.cs](#)

实验包含源生成正则、命名组、**MatchEvaluator**、有限超时和 **NonBacktracking**。

规则:

- 模式固定且高频时考虑 **[GeneratedRegex]**;
- 对不可信输入设置超时;
- 能接受功能限制时, **NonBacktracking** 可提供接近输入长度线性的边界;
- 回溯型结构、反向引用和某些高级特性可能不受非回溯引擎支持;
- 正则适合局部文本语言, 不适合替代完整的递归语法解析器。

练习: 构造逐步增长的对抗输入, 记录回溯引擎耗时与非回溯结果。不要在共享机器上使用无上限输入。

20. 现代密码学原语

20.1 `crypto.modern-primitives`

源码: [ModernCryptographyDemo.cs](#)

先区分用途:

原语	用途
CSPRNG	生成不可预测随机值、salt、nonce、key
SHA-256	无密钥摘要, 不提供身份认证
HMAC-SHA-256	共享密钥消息认证
PBKDF2	从口令和 salt 派生密钥
AES-GCM	加密并认证明文与附加数据
RSA-PSS/ECDSA	私钥签名、公钥验证
FixedTimeEquals	降低早停比较泄漏

实验会篡改 AES-GCM tag 并确认解密被拒绝, 还会清除敏感 key buffer。

必须理解:

- nonce 在同一 AES-GCM key 下必须满足唯一性要求;
- salt 可以公开, 但应随机且按记录保存;
- hash 不是加密, 普通快速 hash 不适合直接存密码;
- 签名不提供机密性;
- 清零托管缓冲区不能保证所有副本、寄存器和内部实现状态都被立刻擦除;
- 生产系统还需要密钥托管、访问控制、轮换、算法协商和威胁建模。

练习: 分别篡改 ciphertext、tag 和 AAD, 确认认证失败; 不要自行设计新的加密协议。

第九部分: 跨语言与原生互操作

21. 托管语言互操作

21.1 `interop.managed-languages`

源码: [ManagedLanguagesInteropDemo.cs](#)

配套项目:

- [F# 项目](#)
- [Visual Basic 项目](#)

C#、F#、VB 共享 CLR、CTS 和程序集元数据, 因此可以直接引用。但语言特有结构会影响公共 API 的 C# 体验。公共边界优先使用稳定、可空性清晰的 CLR 类型; 不要让调用方必须理解另一语言的内部习惯才能安全调用。

练习: 从 F# 返回 option 或 discriminated union, 观察 C# 看到的实际 API 形状, 再设计一个 C# 友好的 facade。

22. 平台 P/Invoke

22.1 `interop.native-pinvoke`

源码: [NativeInteropDemo.cs](#)

LibraryImport 通过源生成建立平台调用。必须核对:

- 目标库和导出符号;
- calling convention;
- 整数宽度与结构布局;
- 字符串编码;
- 错误码/**GetLastError** 规则;
- 内存由谁分配和释放;
- API 是否只存在于特定平台。

平台分支不是“让测试通过”的技巧,而是公共 API 能力的一部分。

23. Python 进程协议

23.1 `interop.python-process-json`

源码: [PythonProcessInteropDemo.cs](#) 与 [interop_worker.py](#)

边界设计:

```
C# host
-> 工作区 .venv python
-> stdin: UTF-8 JSON Lines request
<- stdout: UTF-8 JSON Lines response
<- stderr: diagnostics only
-> timeout / cancellation / exit code / process cleanup
```

优点是 ABI 隔离、部署模型清晰、Python 崩溃不直接破坏 CLR。代价是序列化、进程启动、协议版本和错误映射。

练习:

1. 增加协议版本字段和不支持版本错误。
2. 让 worker 超时, 确认宿主终止并回收子进程。
3. 输出一行非法 JSON, 区分协议错误与非零退出码。

24. C ABI 与反向回调

24.1 `interop.c-abi`

源码: [CAbiInteropDemo.cs](#)、[learn_c.h](#) 与 [learn_c.c](#)

C ABI 边界使用固定宽度类型、blittable struct、指针+长度、整数状态码。**UnmanagedCallersOnly** 允许 C 调用托管函数指针, 但必须保证:

- 委托/函数入口在调用期间有效;
- 不让托管异常穿过原生栈;
- 不越界读取指针;
- 结构布局和 calling convention 完全一致;

- 错误转换为明确状态码。

练习：给结构体增加字段，分别修改一侧和两侧，观察 ABI 版本不一致的风险。

25. C++ 不透明句柄

25.1 interop.cpp-opaque-handle

源码：[CppAbiInteropDemo.cs](#)、[learn_cpp.h](#) 与 [learn_cpp.cpp](#)

不要直接把 C++ class、异常、`std::string`、`std::vector` 暴露给 C#。本实验使用：

- `extern "C"` 导出稳定函数名；
- opaque handle 隐藏 C++ 对象布局；
- 状态码阻止 C++ 异常跨边界；
- 调用方提供 UTF-8 缓冲区；
- `SafeHandle` 确保 C# 侧释放对象；
- 复制 API 把 vector 内容转换为稳定数据。

练习：模拟缓冲区过小，设计“先查询所需长度，再复制”的两阶段 API；确认所有失败路径都不泄漏句柄。

第十部分：综合训练与验收

26. 五个可运行综合项目

前 25 章大多把一个概念边界单独放大。本章反过来：每个项目级实验都让多个边界共同工作，并用不变量证明它们没有被“拼接代码”掩盖。五项仍由统一 CLI 发现，因此既可单独运行，也会进入 `self-test`：

```
dotnet run --project src/LearnDotnetCSharp.App -- run project.cancellable-data-pipeline
dotnet run --project src/LearnDotnetCSharp.App -- run project.versioned-local-service
dotnet run --project src/LearnDotnetCSharp.App -- run project.collectible-plugin-host
dotnet run --project src/LearnDotnetCSharp.App -- run project.polyglot-compute
dotnet run --project src/LearnDotnetCSharp.App -- run
project.resilient-analytics-workflow
```

本轮新增 `src/LearnDotnetCSharp.Capstones`，把需要直接测试的恢复机制与 CLI 展示分开。A -> B 表示 A 依赖 B：

```
LearnDotnetCSharp.App -> LearnDotnetCSharp.Capstones -> LearnDotnetCSharp.PluginContract
LearnDotnetCSharp.Tests -> LearnDotnetCSharp.Capstones
LearnDotnetCSharp.Tests -> LearnDotnetCSharp.App
```

Core 按 `DataPipeline`、`LocalService`、`PluginHost`、`Polyglot` 组织，不引用 `App`、`IDemo`、控制台或 `DemoAssert`。它返回状态、结果或明确异常；App 再接入真实文件、Kestrel、SQLite、工作区 Python、插件 DLL、C/C++ 与可观测性，并把固定输入写成教学验收。学习时不要只看最终输出。先在每个阶段旁写下“谁创建、谁完成、谁取消、谁释放、什么状态已经持久化”，再故意破坏一个不变量，观察错误能否回到拥有者。

26.1 带 checkpoint、死信与恢复的数据管线

实验：`project.cancellable-data-pipeline`

源码：[CancellableDataPipelineProjectDemo.cs](#)、[ResumableDataPipeline.cs](#) 与 [JsonPipelineStateStore.cs](#)

深化后的项目不再只证明“一次运行处理成功”，而是证明物理字节位置、业务提交和持久化恢复点之间的关系：

```
UTF-8 文件 + SHA-256 source fingerprint
-> 从 checkpoint.NextByteOffset 定位
```

- > 按换行拆出带 sequence/start/next byte offset 的物理帧
- > bounded Channel 施加背压
- > 成功帧进入幂等业务 sink
- > 失败帧形成稳定 EntryId 的 dead letter
- > ContiguousCheckpointTracker 只推进连续终态前缀
- > checkpoint + 去重死信原子写入 JSON
- > NDJSON 死信只是可重建投影

这里的语义是 **at-least-once 读取 + 幂等 sink**，不是凭空得到 exactly-once。消费者 4 可能先于消费者 3 完成，但 checkpoint 不能越过 sequence 3 的空洞；**ContiguousCheckpointTracker** 暂存乱序终态，只有连续前缀闭合时才更新字节高水位。状态文件同时保存 checkpoint 与已去重死信，写入临时文件、flush 到磁盘后再原子替换；NDJSON 文件若在投影阶段中断，可以从权威 JSON 快照重建。

项目先以单消费者运行，在 ID 3 的 sink 成功前由 **FaultPlan** 中断；随后创建全新的 runner，以三个消费者从 sequence 3、对应 UTF-8 字节偏移继续。源指纹绑定 checkpoint 与原文件；若内容被替换，即使路径相同也会 fail closed，而不是把旧偏移应用到新数据。

稳定验收为：首轮留下连续 checkpoint 3，续跑处理三个物理帧；最终 5 条合法记录、1 条死信、总值 36，业务 ID 1..5 各提交一次；非法帧跨恢复仍只有一条死信；最终字节偏移等于源文件长度。Activity、Meter 和源生成 JSON 继续验证恢复前后的 trace 与报告契约。

练习：

1. 让 sequence 4 在 gate 后等待、sequence 5 先完成，观察 checkpoint 仍停在 3；释放 gate 后再一次推进到 5。
2. 在业务 sink 已成功、checkpoint 尚未保存的边界中断，解释为何重放不可避免，并把 sink 从内存集合换成带唯一键的 SQLite 表。
3. 删除或截断 dead-letter NDJSON 后重开状态，证明它能从权威 JSON 快照重建且 EntryId 不重复。
4. 修改源文件一个字节后复用旧状态，确认 **PipelineSourceMismatchException** 阻止错误续跑；再设计显式“放弃旧 checkpoint”命令。
5. 将状态格式版本改为未知值，写出迁移策略，不要无条件忽略新旧格式差异。

26.2 可持久化、可重启的版本化协议服务

实验：**project.versioned-local-service**

源码：[VersionedLocalServiceProjectDemo.cs](#) 与 [SqliteIdempotencyStore.cs](#)

项目仍只监听 **IPAddress.Loopback:0** 的 Kestrel HTTP/1.1，不访问公网。客户端先用源生成 JSON 上下文得到**确定的原始 UTF-8 字节**，再计算规范化 HMAC：

```
METHOD
/api/jobs
protocol-version
correlation-id
idempotency-key
nonce
base64(sha256(exact-body-bytes))
```

服务端必须对收到的原始正文验签，不能“反序列化再序列化”后比较；签名用

CryptographicOperations.FixedTimeEquals 比较。认证成功后才记录 nonce，正文同时受

Content-Length 和实际流读取的 32 KiB 上限约束。业务结果不再放在进程内字典，而是交给

SqliteIdempotencyStore：同键同 hash 返回原始 **byte[]**，同键异 hash 返回 conflict；同进程跨 Store 实例的 single-flight 保证并发相同请求只运行一次 factory。

SQLite 每次调用使用独立连接，初始化 WAL 并设置 `busy_timeout`。factory 在事务外运行，成功后才进入短 `BEGIN IMMEDIATE` 事务写入完整响应；取消或异常不会伪造完成行。最关键的故障点故意位于提交之后、发送响应之前：

```
factory 计算 accepted/completed NDJSON bytes
-> SQLite COMMIT 完成
-> FaultPlan 返回 503
-> 客户端不知道服务是否已经提交
-> 新 HttpRequestMessage + 新 nonce + 相同业务幂等键重试
-> 从 SQLite 逐字节重放，不再次运行 factory
```

一次运行还覆盖错误签名 401、不支持版本 426、显式同键重放、同键不同正文 409，以及真正停止第一个 Kestrel、重新创建 Store 和服务状态后的再次重放。`[JsonExtensionData]` 保留 `priority`；关联 ID 贯穿事件。客户端以 `ResponseHeadersRead` 增量读取 NDJSON，协调 gate 证明第一条事件已 flush 时第二条尚未完成。

稳定验收为：首个成功业务只执行 1 次；提交后 503 的自动重试和显式重放均命中持久化响应；相同键不同正文冲突；重启后的新服务执行数为 0、durable replay 为 1；三次成功读取的事件序列逐项相同，结果为 22。

练习：

1. 并发发送 32 个同键同 hash 请求，用 gate 卡住 factory，证明只有一个 `Executed`，其余均为 `Replayed`。
2. 让 factory 抛异常或取消，再以同键重试；查询数据库并证明失败尝试没有留下完成行。
3. 在 SQLite 已提交后模拟连接直接断开，而不是返回 503，确认客户端仍只能依靠幂等键消解提交歧义。
4. 增加 nonce 过期时间和有界 replay cache，说明清理任务、时钟和多实例一致性的所有者。
5. 在正式部署设计中换成 TLS、密钥轮换与真正的跨进程协调；当前 single-flight 是进程内保证，SQLite 唯一键才是持久化事实边界。

26.3 多版本、可热切换的隔离插件宿主

实验：`project.collectible-plugin-host`

源码：`CollectiblePluginHostProjectDemo.cs`、`CollectiblePluginHost.cs`、`PluginRouter.cs`、`SampleLearningPlugin.cs` 与 `AdvancedLearningPlugin.cs`

`LearningPluginAttribute` 仍位于共享契约程序集，声明插件 ID、契约版本和能力。Core 用临时可收集上下文扫描 `CustomAttributeData`，把 ID、契约版本、程序集实现版本、能力、路径和入口类型复制成 `PluginDescriptor`；路由快照不保存插件 `Type`、`Attribute` 实例或 `Assembly`。`PluginRouter` 先匹配契约版本和能力，再排除隔离路径，最后按实现版本降序选择。

实验先只刷新 v1 并执行一次；随后刷新 v1/v2 快照，`text.uppercase` 能力自动热切换到最高兼容的 v2。输入 `fail: deterministic fallback` 让 v2 抛出普通异常：宿主在可收集边界内把异常复制成 `PluginFailure(TypeName, Message)`，请求卸载该 ALC，把 v2 路径加入 quarantine，再用同一逻辑请求回退 v1。隔离期间后续请求直接选择 v1；再次 `Refresh` 建立新快照并解除旧隔离状态，路由恢复 v2。

每次实际尝试使用独立的可收集 `AssemblyLoadContext`。卸载仍是 GC 协议，不是 `Unload()` 的同步承诺：

- 执行 helper 内部持有 `Assembly`、`Type`、实例与真实异常；
- 边界外只返回共享契约结果、复制后的失败描述、描述符和 `WeakReference`；
- 宿主在候选尝试之间检查调用方取消；当前 v1 同步契约无法抢占正在运行的插件，若需要强制超时或隔离死循环，必须把插件放入独立进程；
- helper 在 `finally` 中请求卸载，外层有限轮 GC/finalizer/GC 后记录是否已收集；
- quarantine 保存的是默认上下文字符串路径，不保存插件反射对象。

稳定验收为：更新前 v1；刷新后输出 **V2::AFTER UPDATE**；v2 故障后尝试链为 **v2:failed -> v1:ok**；隔离期间继续使用 v1；再次刷新后回到 v2；六次实际路由尝试的 ALC 全部收集，插件异常对象没有逃出边界。

练习：

1. 新增只提供另一项 capability 的 v2 插件，证明路由由“能力 + 契约版本 + 实现版本”共同决定，而不是只比较文件版本。
2. 把 **failuresBeforeQuarantine** 改为 2，记录第一次失败后的候选集与第二次失败后的隔离快照。
3. 让 v2 抛出包含插件自定义异常类型的异常，确认边界外只剩字符串 **TypeName/Message**，旧 ALC 仍能回收。
4. 故意把插件 **Type** 或原始异常放进静态列表，观察卸载断言失败，再用 GC root 工具定位引用链。
5. 为刷新增加不可变快照交换，让并发读取继续使用旧快照；说明何时才能删除旧插件文件。

26.4 可自愈的跨语言计算模块

实验：**project.polyglot-compute**

源码：[PolyglotComputeProjectDemo.cs](#)、[PythonWorkerPool.cs](#)、[PythonWorkerProtocol.cs](#)、[CAbiInteropDemo.cs](#) 与 [CppAbiInteropDemo.cs](#)

项目外层仍用容量为 1 的有界 Channel 调度四个批次、两个消费者并发处理；Python 边界则从“每请求启动一个进程”深化为两个常驻 session 和容量为 4 的有界工作队列：

```
C# batch
-> PythonWorkerPool bounded queue
-> 两个 workspace .venv 常驻进程 / JSON Lines
-> 请求 ID 关联响应，独立 timeout，持续 drain stderr
-> worker 崩溃时终止进程树、创建 generation+1 session
-> 对声明为可重试的同一幂等分析请求最多重试一次
-> C ABI / pointer + length + blittable struct: count, sum, mean
-> C++ extern "C" / opaque handle / vector history
-> C# ConcurrentDictionary exactly-once + source-generated JSON report
```

batch-a 的第一次 Python 尝试在写响应前按协议确定性退出。池观察 EOF/退出码与有界 stderr tail，替换该 session 后重放同一请求；其他常驻 worker 可继续处理队列。请求 ID 在 in-flight 集合中唯一，避免两个调用竞争同一响应。调用方取消或单请求超时时会替换无法再安全复用的 session；**DisposeAsync** 完成队列、取消循环并清理全部进程树。

C 阶段仍只在同步 **unsafe** helper 内固定数组，不跨 **await** 保存指针；C++ 阶段仍为每个批次创建独立句柄，并由 **SafeHandle.Dispose** 关闭。稳定验收为：四批恰好一次；Python 池 **Starts=3**、**Restarts=1**，结束时两个活跃 PID、总共观察三个 PID；每批 Python/C 的 count、sum、mean 相同；C++ history 为逐项前缀和且句柄关闭。完整 C/C++ 链路当前需要 Windows x64，其他平台明确 **Skipped**；Python worker 池本身由跨平台正式测试覆盖。

练习：

1. 提交带 **DelayMilliseconds** 的慢请求，在 gate 确认其已进入 worker 后取消；证明旧 PID 消失、替代 session 出现且下一请求成功。
2. 同时提交两个相同 request ID，确认第二个在写入队列前被拒绝；再设计跨进程全局 ID 的生成规则。
3. 把 **MaxCrashRetries** 改为 0 和 1，分别观察异常传播与自动恢复；解释为什么非幂等操作不能盲目重放。
4. 将批次增加到 100，比较 worker 数、队列容量、吞吐、尾延迟和进程隔离成本，不用一次 Stopwatch 输出下结论。
5. 故意让 C 结构布局或 C++ 缓冲区容量不匹配，确认错误以状态码到达，而不是让异常跨 ABI。

6. 为每批增加 Activity，把同一协议 ID 写入 Python 请求、C# trace、原生日志和指标标签。

26.5 可恢复的跨语言分析 workflow

实验：[project.resilient-analytics-workflow](#)

源码：[ResilientAnalyticsWorkflowProjectDemo.cs](#)

这是第五个、也是最终端到端项目。它不要求仓库 C/C++ DLL，因此在 Windows 与 Linux 上都可以验证托管恢复链；前提仍是先用 `setup-python.cmd` 或 CI 创建工作区 `.venv`。同一请求依次穿过四个 Capstones 边界：

```
SQLite idempotency key + source SHA-256
-> ResumableDataPipeline<AnalyticsMetric>
  -> 原子 checkpoint / 一条幂等 dead letter
  -> 故障点在 metric 2 的 sink 之前中断
-> 同一 factory 第二次执行，从连续 checkpoint 恢复
-> PythonWorkerPool 分析 [4, 7, 11]
  -> 第一次 worker 在响应前崩溃
  -> generation+1 session 重试，得到 sum=22
-> CollectiblePluginHost 按 text.uppercase 能力选择 v2
-> 源生成 JSON 形成最终 byte[]
-> SQLite 提交完成
-> Dispose Store，重新打开数据库并逐字节 replay
```

这里有两套彼此独立的恢复状态。第一次 workflow factory 抛出 `AnalyticsWorkflowInterruptedException` 时，SQLite 不保存“已完成响应”，但数据管线自己的 JSON checkpoint 和死信已经保存连续终态前缀；第二次 factory 因而可以 resume。Python worker 的 generation、PID 和请求重试属于进程池的瞬时状态；插件描述符和 quarantine 属于路由快照；只有最终响应进入 SQLite 后，整个请求才达到 durable complete。

当前示例用 `ConcurrentDictionary` 作为同一进程内的幂等业务 sink，让两次 runner 执行不会重复提交 metric。它演示恢复契约，但不声称这个内存集合能跨进程持久化；若要模拟真正进程重启，应把业务 sink 也换成带唯一业务键的持久化存储。

稳定验收为：workflow factory 恰好运行 2 次，数据库重开后的 replay factory 为 0 次；最终 `Executed -> Replayed` 且响应字节完全相同；3 条合法 metric、1 条死信、checkpoint sequence 4、sum 22；Python 池 `Starts=3、Restarts=1`；插件版本为 2.x、输出以 `V2::` 开头，并且执行 ALC 已收集。

修改练习：

1. 把管线故障移到业务 sink 成功之后、checkpoint 保存之前。先观察内存 sink 如何去重，再把它换成 SQLite 唯一键并真正重建进程级对象。
2. 第一次执行后修改源文件一个字节，继续使用旧 checkpoint 和幂等键；分别说明 source fingerprint 与 request hash 在哪一层拒绝不一致。
3. 让 v2 对最终格式化输入抛异常，确认 workflow 使用 v1 fallback 仍能完成；再检查所有失败尝试的 ALC 是否回收。
4. 把 Python `MaxCrashRetries` 设为 0，证明最终响应不会写入 SQLite；恢复配置后同键仍能执行成功。
5. 在 SQLite 已提交后、调用方收到结果前注入异常，重新打开 Store 并 replay，比较它与 26.2 “提交后 503”的共同提交歧义。
6. 为 workflow 外层增加 loopback HTTP 和 NDJSON 进度事件，把 correlation ID 传播到 pipeline、Python 请求、插件 Activity 与最终持久化响应。

27. 结业问答

如果不能独立回答，回到相应实验：

1. C# 语言版本、TFM、SDK 和运行时版本分别控制什么？
2. 为什么 `async` 不等于创建线程？
3. `IEnumerable<T>` 与 `IQueryable<T>` 的执行边界在哪里？
4. `AsEnumerable` 为什么可能造成严重的数据访问问题？
5. TCP 为什么需要 framing，而 UDP 为什么仍可能需要消息 ID？
6. HTTP 请求版本与实际响应版本为什么可能不同？
7. `Span<T>`、`Memory<T>` 和数组的所有权关系是什么？
8. 为什么拥有 `SafeHandle` 的类通常不需要 `finalizer`？
9. `Activity`、`Metric`、`Log` 和 `EventSource` 分别回答什么问题？
10. `AssemblyLoadContext.Unload()` 为什么不代表已经卸载？
11. expression tree、反射调用、`Emit` 和 `Roslyn` 编译有什么差异？
12. 为什么正则超时与 `NonBacktracking` 是两种不同的复杂度策略？
13. hash、HMAC、加密和签名的安全目标有什么不同？
14. JSON 多态为什么需要白名单判别器？
15. 为什么 C++ 对外提供 C ABI 外壳，而不是直接导出 STL 类型？
16. `Python venv` 解决了什么，又没有解决什么？
17. 协变、逆变和不变分别允许哪些安全转换，为什么值类型不参与引用型变？
18. `SynchronizationContext` 与 `ExecutionContext` 分别控制 continuation 的什么方面？
19. CAS 循环的线性化点在哪里，ABA 为什么不能仅靠“值又相等”发现？
20. `ResponseHeadersRead` 后为什么必须为正文读取单独管理取消和生命周期？
21. 固定托管对象与分配原生内存的所有权、GC 和释放责任有什么不同？
22. 增量源生成器与诊断分析器分别可以改变或报告什么？
23. 为什么一次预热或 `Stopwatch` 输出不能证明动态 PGO/SIMD 带来了性能提升？
24. 并发消费者乱序完成时，`checkpoint` 为什么只能推进连续终态前缀？
25. 服务在提交后断开或返回 503 时，客户端为什么必须重用业务幂等键？
26. 插件异常为什么要复制成默认上下文数据，而不能把原始异常对象返回宿主？
27. `Python` 常驻 worker 崩溃后，哪些请求允许重试，哪些请求必须交给上层决策？

28. 最终验收清单

- `self-test` 的 `Failed` 与 `Timeout` 均为 0；当前平台支持的实验全部 `Passed`，不支持项明确 `Skipped`。
- 每个实验至少完成一次“预测 - 运行 - 修改 - 复盘”。
- 能画出异步取消和并发管线的拥有者关系。
- 能解释 TCP、UDP、WebSocket、TLS、HTTP/2、HTTP/3 的层次。
- 能解释 GC、Dispose、SafeHandle、finalizer 的责任分工。
- 能从 C# 源码找到对应 IL 或生成状态机。
- 能展示一个真实 EF Core 查询的表达式树与 SQL。
- 能设计一个可演进且限制输入的序列化契约。
- 能为 Python/C/C++ 边界写出协议、ABI 与所有权说明。
- 能说明密码学示例之外仍需哪些生产安全措施。

- 五个综合项目都至少完成一次修改，并能解释管线中断、提交后 503、插件失败回退和 Python 崩溃各自的恢复状态。
- 能画出 App -> Capstones -> PluginContract 依赖边界，并说明 30 个正式测试与 53 项 self-test 分别证明什么。

附录 A：实验速查表

分类	实验 ID
runtime	runtime.overview、runtime.jit-simd-pgo、runtime.collectible-plugin
language	language.csharp14、language.advanced、language.operators-conversions、language.generics-variance-constraints、language.attributes、language.delegates-events-iterators
framework	framework.text-globalization、framework.time-numeric-identity
collections	collections.core-custom-buffer
linq	linq.query-semantics、linq.operators、linq.efcore-sqlite-provider
xml/serialization	xml.linq、serialization.xml-json、serialization.contract-versioning
async	async.cancellation-stream-valuetask、async.task-when-each、async.continuations-context
threading	threading.execution-context、threading.synchronization-primitives
concurrency	concurrency.channel-pipeline、concurrency.parallel-plinq、concurrency.memory-model-lock-free
io	io.streams-pipelines-compression、io.random-access-memory-map
networking	networking.tcp-loopback、networking.httpclient-loopback、networking.http2-http3、networking.http-streaming-resilience、networking.udp-websocket-tls
memory	memory.gc、memory.span-memory-ownership、memory.disposal-finalization、memory.pinning-native-memory
diagnostics	diagnostics.observability
reflection/compiler	reflection.advanced、compiler.roslyn-il、compiler.incremental-generator-analyzer
regex/crypto	regex.advanced、crypto.modern-primitives
interop	interop.managed-languages、interop.native-pinvoke、interop.python-process-json、interop.c-abi、interop.cpp-opaque-handle
projects	project.cancellable-data-pipeline、project.versioned-local-service、project.collectible-plugin-host、project.polyglot-compute、project.resilient-analytics-workflow

附录 B：性能实验纪律

本仓库不是微基准框架。需要做性能结论时：

1. 使用 Release；
2. 不附加调试器；
3. 预热；
4. 多轮测量并报告分布；
5. 固定输入和环境；
6. 记录 GC、CPU、OS、运行时版本；
7. 使用 BenchmarkDotNet 等专业工具；
8. 区分吞吐、延迟、分配和尾延迟；
9. 不从一次 Stopwatch 输出推导普遍结论；
10. 优化后重新运行语义断言。

附录 C：从 Markdown 生成 PDF

仓库提供了固定版本、可重复执行的 PDF 构建链。它不依赖 Word、浏览器、Pandoc 或完整 LaTeX 发行版；在仓库根目录运行：

```
.\scripts\build-study-guide-pdf.cmd
```

构建流程会：

1. 通过工作区 `.venv` 创建独立的 `.docs-venv`；
2. 按 `requirements-docs.txt` 安装精确锁定的 ReportLab、PyPDF、Pillow 和字符编码依赖；
3. 自动寻找 Microsoft YaHei、Noto CJK、WenQuanYi 等中文字体，以及 Cascadia Mono、DejaVu Sans Mono 等代码字体；
4. 把字体嵌入 PDF，渲染 YAML 元数据、目录、三级标题、列表、引用、表格、代码块和链接；
5. 把仓库内相对链接改写为 GitHub `main` 分支链接；
6. 写入 PDF 书签、页脚和页码；
7. 自动验证 A4 页面、文档标题、书签、乱码，并核对源码目录与学习指导中的全部 53 个实验 ID。

第一次运行需要联网下载固定依赖；依赖版本验证通过后，后续构建不会再次访问包索引。输出文件是：

```
output\pdf\LearnDotnetCSharp-Study-Guide.pdf
```

本地安装器优先使用清华 PyPI 镜像并以官方 PyPI 作为回退，避免继承用户级 `pip.ini` 中失效的索引地址。推送本文件、生成器或依赖清单到 `main` 时，`.github/workflows/build-study-guide-pdf.yml` 会在 Ubuntu 上安装带 TrueType 轮廓的文泉驿正黑字体，运行同一生成器，并上传保留 30 天的 `LearnDotnetCSharp-Study-Guide` artifact；也可从 Actions 页面手动运行该工作流。生成器会依次尝试候选字体并跳过 ReportLab 无法注册的 CFF/PostScript 轮廓字体。

若自动字体探测不适合当前系统，可显式指定字体：

```
.\scripts\build-study-guide-pdf.cmd `
--font-regular "C:\Windows\Fonts\msyh.ttc" `
--font-bold "C:\Windows\Fonts\msyhbd.ttc" `
--font-code "C:\Windows\Fonts\CascadiaMono.ttf"
```

若在 CI 中已经准备了满足 `requirements-docs.txt` 的 Python, 可跳过 `.docs-venv` 的创建:

```
$env:LEARN_DOTNET_DOCS_PYTHON = "C:\path\to\python.exe"
.\scripts\build-study-guide-pdf.cmd
```

生成器会做结构性检查; 发布前仍建议使用 Poppler 做一次渲染抽查:

```
pdftinfo .\output\pdf\LearnDotnetCSharp-Study-Guide.pdf
New-Item -ItemType Directory -Force .\tmp\pdfs | Out-Null
pdftoppm -png .\output\pdf\LearnDotnetCSharp-Study-Guide.pdf .\tmp\pdfs\study-guide
```

检查封面、目录、表格密集页、长代码页和最后一页, 确认没有裁切、孤立标题或缺字。

`output/pdf/LearnDotnetCSharp-Study-Guide.pdf` 作为便于直接阅读的主线快照纳入版本控制; 此 Markdown 仍是可审查的内容源。修改正文后, 应重新生成 PDF, 并把 Markdown 与 PDF 放在同一个提交中。

附录 D: 延伸资料

本书不复制规范和 API 文档。继续学习时从 [Microsoft 官方延伸资料](#) 出发, 重点阅读:

- .NET 10、C# 14、泛型约束/型变与静态抽象接口成员;
- LINQ 与 EF Core 10 查询翻译;
- async 上下文、内存模型、Channels、Pipelines 与 HTTP 流/韧性;
- GC、固定/原生内存、Dispose、诊断、JIT/SIMD/PGO 与 AssemblyLoadContext;
- Roslyn 生成器/分析器、正则、密码学和原生互操作。

阅读官方资料时, 把每个规则映射回一个可运行实验; 阅读实验时, 把每个行为映射回官方契约。两者互相校验, 才能避免把偶然输出当成语言或运行时保证。